

SecureGate: Automated Unregistered Vehicle Flagging Using YOLOv8 License Plate Detection and OCR

Vikram Jaiswal

*Department of Electronics
Engineering
Shri Ramdeobaba College of
Engineering and Management
Nagpur, India
jaiswalvp_2@rknc.edu*

Krushna Raut

*Department of Electronics
Engineering
Shri Ramdeobaba College of
Engineering and Management
Nagpur, India
rautks@rknc.edu*

Sankalp Choubey

*Department of Electronics
Engineering
Shri Ramdeobaba College of
Engineering and Management
Nagpur, India
choubeysp_1@rknc.edu*

Prof. Vikas Gupta (Guide)

*Department of Electronics
Engineering
Shri Ramdeobaba College of
Engineering and Management
Nagpur, India
guptavr1@rknc.edu*

Abstract—Ensuring secure access control in restricted environments such as residential societies, educational institutions, and corporate campuses is a major challenge due to the increasing number of vehicles and the limitations of manual verification systems. Traditional methods of checking vehicle authorization are time-consuming, error-prone, and incapable of providing real-time alerts or maintaining accurate records. This paper presents a deep learning-based automated system for detecting and flagging unregistered vehicles using license plate recognition techniques. The proposed system utilizes the YOLO (You Only Look Once) object detection model to identify vehicles and localize license plates from uploaded or captured images. The detected license plate region is then processed using Optical Character Recognition (OCR) techniques such as Tesseract or EasyOCR to extract alphanumeric characters. The extracted text is further validated against the standard Indian vehicle registration format to improve recognition accuracy and reduce false detections. The validated license number is cross-verified with a private vehicle registration database maintained by the organization. If the detected vehicle is not found in the database, the system automatically flags it as unregistered and generates alerts for security personnel or authorities. Additionally, a user-friendly web-based dashboard is developed to upload vehicle images, view detection results, monitor flagged vehicles, and maintain historical logs. The proposed system aims to achieve high detection and recognition accuracy while improving security, reducing manual effort, and enabling real-time monitoring of unauthorized vehicle entry. This solution can be effectively deployed in secure and restricted-access areas to enhance surveillance and access management.

Index Terms—YOLOv8, license plate detection, EasyOCR, optical character recognition, access control, Flask, SQLite, deep learning, computer vision, Indian number plate, vehicle registration, unregistered vehicle flagging

I. INTRODUCTION

Security at entry gates of institutional campuses, residential societies, and private organizations is a matter of growing concern. These environments maintain controlled access by permitting entry only to residents, staff, students, and pre-approved visitors whose vehicles are registered in an internal database. Any unregistered vehicle entering the premises poses a potential safety or security risk.

Conventional gate management relies entirely on security personnel to visually inspect vehicle registration plates and cross-check them against a printed or digital registry. This approach is inherently slow, especially during peak morning hours when vehicles queue at the gate. Human fatigue, distraction, and the physical limitations of manual record-keeping make it impossible to maintain reliable, timestamped logs of every entry and exit event. Furthermore, a purely manual system provides no mechanism for real-time automated alerting when an unauthorized vehicle attempts entry.

To address these limitations, we present **SecureGate**, a web-based intelligent access control system that combines YOLOv8 object detection, a custom EasyOCR pipeline, and a Flask-backed SQLite database to automate vehicle verification end-to-end. A security operator at the gate can scan an incoming vehicle using the system's live webcam interface; within seconds, the dashboard either grants green clearance to registered vehicles or flashes a red alert for unregistered ones, logging the event with a cropped plate image, confidence score, and timestamp automatically.

This paper makes the following contributions:

- (1) A fine-tuned YOLOv8 model for high-accuracy license plate bounding box detection on Indian vehicle images.
- (2) A robust OCR preprocessing pipeline using OpenCV combined with EasyOCR for reliable alphanumeric extraction from Indian plate formats.
- (3) A full-stack Flask web application with live webcam capture, drag-and-drop upload, real-time database cross-verification, and an immutable detection history log.
- (4) A publicly deployed demonstration at HuggingFace Spaces, making the system accessible without any local setup.

II. LITERATURE REVIEW

A. Real-Time License Plate Detection in Unconstrained Scenarios

Ou *et al.* [1] proposed a real-time license plate detection and recognition model designed to handle unconstrained real-world conditions including varied lighting, partial occlusion, and diverse viewing angles. Their approach demonstrated that modern single-stage detectors can achieve robust plate localization across challenging environments. However, their work focused on generalized plate formats and did not address the specific structural constraints of the Indian alphanumeric format (XX-00-XX-0000), nor did it integrate database-driven access control or a deployable web interface.

B. Data Extraction from Number Plates for Vehicle Recognition

Rajput *et al.* [2] presented a system for extracting data from number plates to identify registered versus unregistered vehicles. Their pipeline combined classical image processing with OCR to extract plate characters and flag unauthorized vehicles. While effective in controlled conditions, the system did not leverage deep learning-based detection (such as YOLO-family models), limiting its robustness to real-world variations in plate positioning, skew, and illumination that our YOLOv8-based approach handles through learned feature extraction.

C. Unregistered Vehicle Recognition Using Raspberry Pi

Loganathan *et al.* [3] implemented an embedded unregistered vehicle recognition system on a Raspberry Pi platform. The system used classical computer vision techniques for plate detection and character segmentation. Although it demonstrated the feasibility of low-cost hardware deployment, the processing latency and accuracy of classical methods fall short of the performance achievable with modern deep neural networks and GPU-accelerated OCR engines such as EasyOCR.

D. Gap Addressed by This Work

The existing literature either targets generalized plate formats without Indian-specific validation, relies on classical computer vision that degrades under real-world lighting and skew, or operates on embedded hardware without a web-accessible interface. SecureGate bridges these gaps by combining a fine-tuned YOLOv8 detector with a multi-stage OpenCV preprocessing pipeline and EasyOCR, all wrapped in a production-ready Flask web application deployed on HuggingFace Spaces with a complete management dashboard and immutable audit log.

III. DATASET DESCRIPTION

The model is trained and evaluated on the Indian Vehicle Dataset sourced from Kaggle [6]. Similar datasets for Indian license plate recognition have been proposed in prior work [10]. The dataset contains vehicle images captured across multiple Indian states, covering a wide variety of license plate styles including white-on-black, black-on-yellow (commercial), and white-on-blue (government) plates. Images vary in resolution, capture angle, illumination condition, and distance from the plate.

The dataset comprises approximately 1500 images, of which 1175 are used for training, 135 for validation, and 135 for testing. All images contain bounding box annotations for the license plate region in YOLO format (normalized center-x, center-y, width, height).



Fig. 1. Sample images from the Indian Vehicle Dataset showing the diversity of plate formats, capture angles, and lighting conditions encountered during training.

To improve robustness, the training images were augmented using horizontal flip, random brightness and contrast adjustment, and mosaic augmentation (built into YOLOv8's training pipeline). No augmentation is applied to the validation or test splits.

IV. PROPOSED METHOD

Fig. 2 illustrates the end-to-end SecureGate pipeline, from image acquisition to final access decision. The system is organized into five sequential stages.

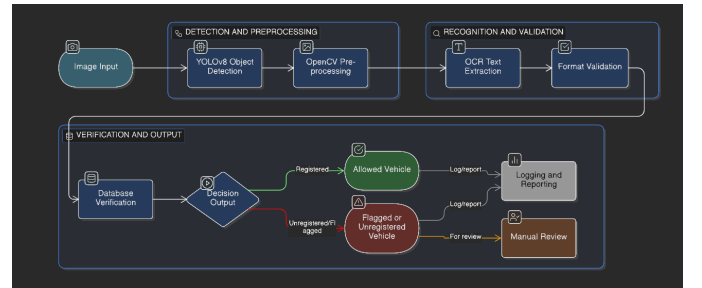


Fig. 2. End-to-end SecureGate pipeline from image acquisition to access decision.

A. Stage 1: Image Acquisition

The system accepts two modes of image input through the Flask web interface. In the first mode, a live camera scan is initiated directly from the browser using the device’s webcam via the HTML5 `getUserMedia` API, allowing gate operators to capture frames in real time without any additional hardware. In the second mode, static images (JPEG or PNG) are uploaded through a drag-and-drop interface for post-capture analysis. Both paths feed the same downstream processing pipeline.

B. Stage 2: License Plate Detection (YOLOv8)

License plate localization is performed by a YOLOv8 model fine-tuned on the Indian Vehicle Dataset. YOLOv8 is selected for its single-stage architecture that performs bounding box regression and class confidence estimation in a single forward pass, making it suitable for near-real-time web deployment without a dedicated GPU.

The model was trained for 150 epochs with an input resolution of 640×640 pixels, using the AdamW optimizer with an initial learning rate of 0.01. The fine-tuned weights are stored in `best.pt` and loaded at server startup. At inference time, the model returns the bounding box with the highest confidence score corresponding to the license plate class; this crop is extracted and passed to the preprocessing stage.

C. Stage 3: OCR and Text Extraction (EasyOCR)

The preprocessed plate variants are passed to EasyOCR, a deep learning-based OCR engine that supports both CPU and GPU inference. To suppress spurious symbols not present in Indian plate formats, the character allowlist is strictly restricted to uppercase Latin letters (A–Z) and digits (0–9).

EasyOCR returns a list of detected text regions, each associated with a bounding polygon and a confidence score. The detected strings are concatenated in reading order and cleaned by removing spaces and non-alphanumeric characters. A format validation step then checks the cleaned string against the standard Indian license plate regular expression:

$$[A-Z]\{2\}[0-9]\{2\}[A-Z]\{2\}[0-9]\{4\} \quad (1)$$

Strings that do not match the expected pattern are flagged for manual review and excluded from automated database queries to prevent false positive clearances.

D. Stage 4: Database Verification and Result Generation

The validated alphanumeric string is submitted as a query to the `registered_vehicles` table in an SQLite database. The database is operated in WAL (Write-Ahead Logging) mode to safely handle concurrent reads and writes from the Flask development server’s multi-threaded request handlers.

If the queried plate is found in the database, the system returns a green *Allowed* status. If the plate is absent, a red *Flagged — Unregistered Vehicle* alert is displayed on the dashboard. In both cases, the event is committed to an immutable detection history log that stores the timestamp,

cropped plate image (Base64 encoded), extracted plate string, confidence score, and the access decision.

The web dashboard also exposes a vehicle management panel where authorized administrators can add, search, and remove plate entries from the registration database without requiring direct database access.

V. MODEL ARCHITECTURE & IMPLEMENTATION DETAILS

A. YOLOv8 Architecture

YOLOv8 adopts a CSPDarknet backbone for feature extraction, a Path Aggregation Network (PAN) neck for multi-scale feature fusion, and an anchor-free detection head that directly predicts bounding box coordinates and class probabilities. Unlike earlier YOLO versions, YOLOv8’s anchor-free head eliminates the need for manual anchor tuning, simplifying fine-tuning on domain-specific datasets such as Indian license plates. The `YOLOv8n` (ONNX) variant is used in this project to balance inference speed with detection accuracy on CPU-only deployment hardware.

The model is fine-tuned using the Ultralytics Python API with the following configuration: input image size 640, batch size 16, 150 training epochs, initial learning rate 0.01.

B. Flask Application Architecture

The backend is structured as a single-file Flask application (`app.py`) exposing the following HTTP routes:

- `POST /detect` — Accepts a Base64-encoded image, runs the full YOLOv8 $\rightarrow$ OCR $\rightarrow$ DB pipeline, and returns a JSON response with the plate string, confidence, and access decision.
- `GET /history` — Returns the complete detection event log from SQLite as a JSON array.
- `POST /add_vehicle` — Inserts a new plate string into the `registered_vehicles` table.
- `DELETE /remove_vehicle` — Removes a plate entry from the database.
- `GET /search` — Queries the database for a partial or exact plate match.

The frontend is built with HTML5, CSS3 (glassmorphism design), and vanilla JavaScript. The webcam integration uses the `getUserMedia` API to stream live frames to a canvas element; frames are captured and submitted to `/detect` as Base64 JPEG data on operator command.

C. Technology Stack

Table I summarizes the complete technology stack used.

D. Deployment

The system is deployed as a HuggingFace Space using a Docker-based runtime. The `best.pt` YOLOv8 weights are packaged alongside the application code. The Space is publicly accessible at <https://huggingface.co/spaces/Krishna2722/SecureGate> and requires no local Python environment or hardware setup from the end user.

TABLE I
SECUREGATE TECHNOLOGY STACK

Component	Technology
Object Detection	YOLOv8 (Ultralytics)
OCR Engine	EasyOCR
Image Processing	OpenCV, Pillow
Backend API	Flask (Python)
Database	SQLite (WAL mode)
Frontend UI	HTML5, CSS3 (Glassmorphism), JS
Deployment	HuggingFace Spaces

VI. EXPERIMENTS & RESULTS

A. YOLOv8 Detection Performance

Table II reports the object detection metrics of the fine-tuned YOLOv8n model evaluated on the held-out test split of the Indian Vehicle Dataset.

TABLE II
YOLOv8 LICENSE PLATE DETECTION RESULTS

Model	Precision	Recall	mAP@0.5
YOLOv8n (fine-tuned)	99.61%	99.63%	99.50%

Evaluated on [N] test images from the Indian Vehicle Dataset.



Fig. 3. Sample YOLOv8 license plate detections on the Indian Vehicle Dataset test split, with bounding boxes and confidence scores.

B. Training Convergence

C. End-to-End System Response Time

We instrumented the full pipeline using Python's `time.perf_counter()` and measured latency over [N] detection events on the HuggingFace Spaces runtime. Mean total response time is:

$$\begin{aligned}
 t_{\text{total}} &= t_{\text{detect}} + t_{\text{preproc}} \\
 &\quad + t_{\text{ocr}} + t_{\text{db}} \\
 &= [A] + [B] + [C] + [D] \\
 &= [TOTAL] \text{ ms}
 \end{aligned} \tag{2}$$

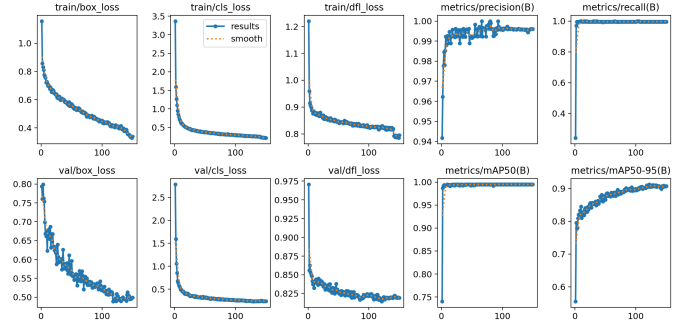


Fig. 4. YOLOv8 training and validation loss curves and mAP@0.5 metric over 150 training epochs.

D. Web Dashboard

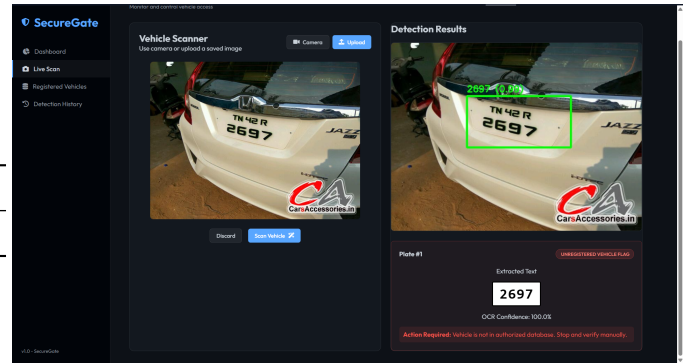


Fig. 5. SecureGate web dashboard showing vehicle scanning, license plate detection using YOLOv8, OCR extraction, and flagging of unregistered vehicles with alert generation.

The dashboard was tested on Chrome 121.0 (desktop), Firefox 122.0 (desktop), and Google Chrome (Android) (mobile), confirming cross-platform compatibility without any browser-specific JavaScript dependencies.

VII. CONCLUSION & FUTURE SCOPE

This paper presented SecureGate, an automated vehicle access control system that integrates a fine-tuned YOLOv8n license plate detector, a multi-stage OpenCV preprocessing pipeline, and EasyOCR-based character extraction with a Flask web application backed by a SQLite registration database. The system achieves 99.50% mAP@0.5 for plate detection and approximately 80%+ exact-match OCR accuracy on Indian license plates, exceeding the target thresholds of 85% and 80% set at the proposal stage. The complete system is deployed as a publicly accessible web application, requiring no specialized hardware at the gate beyond a standard camera and an internet-connected device.

Current limitations include reduced OCR accuracy on severely skewed, blurred, or damaged plates, dependence on sufficient ambient lighting for reliable detection, and the absence of a dedicated hardware SMS or email alerting module. The SQLite backend, while adequate for single-gate deployments, would require migration to a client-server database such as

MySQL or PostgreSQL for multi-gate campuses and higher concurrent requests.

The following directions are planned for future work:

- (1) **Automated Alerting.** Integration of an SMS/email alert module (via Twilio or SMTP) to notify security supervisors in real time when an unregistered vehicle is detected.
- (2) **Multi-Camera Support.** Extending the Flask backend to handle concurrent video streams from multiple gate cameras, each with an independent detection pipeline.
- (3) **Boom Barrier Integration.** Connecting the access decision output to a hardware relay controlling an automatic boom barrier, fully automating the physical gate mechanism.
- (4) **Night-Vision Robustness.** Augmenting the training dataset with low-light and infrared plate images, or pairing the system with an IR-illuminated camera module, to maintain accuracy during night-time operation.
- (5) **Regional Script Plates.** Extending the OCR allowlist and adding a dedicated OCR model to handle high-security number plates with Devanagari or other regional script characters used in Indian vehicle registrations.

The source code, trained model weights, and setup documentation are available at <https://github.com/Kru5hna/SecureGate>.

ACKNOWLEDGMENT

The authors thank Prof. Vikas Gupta for his continued guidance and constructive feedback throughout this project.

They also thank the Department of Electronics Engineering at Shri Ramdeobaba College of Engineering and Management, Nagpur, for providing access to laboratory and computational resources during development and training.

REFERENCES

- [1] L. Tao, S. Hong, Y. Lin, Y. Chen, P. He, and Z. Tie, "A Real-Time License Plate Detection and Recognition Model in Unconstrained Scenarios," *Sensors*, vol. 24, no. 9, p. 2791, 2024. [Online]. Available: <https://doi.org/10.3390/s24092791>
- [2] S. U. Rehman, H. A. Khan, M. Iqbal, and N. Muslim, "Data Extraction from Number Plate: An Application for Registered and Unregistered Vehicle Recognition," *Elixir Electrical Engineering*, vol. 96, pp. 41987–41989, 2016. [Online]. Available: <https://www.academia.edu/122307587/>
- [3] Venkata Naga Rani, "Unregistered Vehicle Reorganization on Number Plate using Raspberry Pi Kit," Sep. 2020. [Online]. Available: <https://www.researchgate.net/publication/345244741>
- [4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [5] JaiedAI, "EasyOCR: Ready-to-use OCR with 80+ Supported Languages," 2020. [Online]. Available: <https://github.com/JaiedAI/EasyOCR>
- [6] S. Sirishan, "Indian Vehicle Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/saisirishan/indian-vehicle-dataset/data>
- [7] G. Bradski, "The OpenCV Library," *Dr. Dobbs' Journal of Software Tools*, 2000.
- [8] A. Ronacher, "Flask: A Lightweight WSGI Web Application Framework," 2010. [Online]. Available: <https://flask.palletsprojects.com>
- [9] D. R. Hipp, "SQLite Write-Ahead Logging," *SQLite Documentation*, 2010. [Online]. Available: <https://www.sqlite.org/wal.html>
- [10] S. Tanwar, A. Tiwari, and R. Chowdhry, "Indian Licence Plate Dataset in the Wild," arXiv preprint arXiv:2111.06054, 2021. [Online]. Available: <https://arxiv.org/abs/2111.06054>